|UNIVERSIDAD MEXIQUENSE DEL BICENTENARIO
UNIDAD DE ESTUDIOS SUPERIORES JIQUIPILCO
Ingeniería en Sistemas Computacionales

DIAGNÓSTICO Y RESOLUCIÓN DE CONFLICTOS SINTÁCTICOS CON GNU BISON

TIPO DE TRABAJO

Practica

ASIGNATURA

Programación de Sistemas

PRESENTA

Diego Deciderio García

Matrícula 10240041

DOCENTE

Mtro. Eduardo Quirino Sanroman

GRUPO	SEMESTRE	FECHA
10SC151	Quinto	05/11/2026

Tabla de contenido

Preparación del entorno y estructura 3
Construcción del analizador inicial (*parser.y*) 3
Construcción del leer (*lexer.l*) 5
Diagnostico inicial: 5 conflictos shift/reduce 7
Corrección de operadores: precedencia y asociatividad 8
Corrección del dangling else: gramática emparejada/no emparejada 11
Validación estricta y compilación final 14
Pruebas obligatorias: validación del parser 17
Archivos de prueba 17
pruebas/01_precedencia.txt 17
pruebas/02_parentesis.txt 17
pruebas/03_else_cercano.txt 17
pruebas/04_else_doble.txt 18
pruebas/05_operador_invalido.txt 18
pruebas/06_else_sin_if.txt 18

Preparación del entorno y estructura

Objetivo

Crear la carpeta de la actividad, los archivos base, el .gitignore y registrar las versiones de las herramientas.

Comandos ejecutados

```
Equipo1@TechNova-Group:~$ cd ~/compint/actividades
Equipo1@TechNova-Group:~/compint/actividades$ mkdir -p actividad05_conflictos/pruebas actividad05_conflictos/evidencias
Equipo1@TechNova-Group:~/compint/actividades$ cd actividad05_conflictos
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ pwd
/home/Equipo1/compint/actividades/actividad05_conflictos
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ touch parser.y lexer.l ejecutar_pruebas.sh README.md .gitignore
touch pruebas/01_precedencia.txt \
      pruebas/02_parentesis.txt \
      pruebas/03_else_cercano.txt \
      pruebas/04_else_doble.txt \
      pruebas/05_operador_invalido.txt \
      pruebas/06_else_sin_if.txt
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ cat > .gitignore << 'EOF'
parser.tab.c
parser.tab.h
lex.yy.c
parser
parser.exe
*.o
EOF
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ bison --version | head -n 1 > evidencias/00_versiones.txt
flex --version | head -n 1 >> evidencias/00_versiones.txt
gcc --version | head -n 1 >> evidencias/00_versiones.txt
git --version >> evidencias/00_versiones.txt
cat evidencias/00_versiones.txt
bison (GNU Bison) 3.8.2
flex 2.6.4
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
git version 2.53.0
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ ls -R
.:
README.md  ejecutar_pruebas.sh  evidencias  lexer.l  parser.y  pruebas

./evidencias:
00_versiones.txt

./pruebas:
01_precedencia.txt  03_else_cercano.txt  05_operador_invalido.txt
02_parentesis.txt   04_else_doble.txt    06_else_sin_if.txt
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$
```

Ilustración 1 Comandos ejecutados

Construcción del analizador inicial (*parser.y)*

Objetivo

Escribir la gramática sin precedencia ni reescritura del else, para provocar de forma controlada los 5 conflictos shift/reduce.

```
%require "3.7"
%{
#include <stdio.h>

int yylex(void);
void yyerror(const char *mensaje);
extern int yylineno;
%}

%define lr.type lalr
%define parse.error detailed
%define parse.lac full

%token ID "identificador" NUM "numero"
%token SI "if" SINO "else"
%token ASIGNA "=" MAS "+" POR "*"
%token PAR_I "(" PAR_D ")" PUNTO_COMA ";"

%start programa

%%
programa:
    %empty
  | programa sentencia
  ;

sentencia:
    ID ASIGNA expr PUNTO_COMA
  | SI PAR_I expr PAR_D sentencia
  | SI PAR_I expr PAR_D sentencia SINO sentencia
  ;

expr:
    ID
  | NUM
  | expr MAS expr
  | expr POR expr
  | PAR_I expr PAR_D
  ;
%%

void yyerror(const char *mensaje)
{
    fprintf(stderr, "Error sintactico en la linea %d: %s\n",
            yylineno, mensaje);
}

```

```
48  int main(void)
49  {
50      return yyparse();
51  }
```

Ilustración 2 Código completo

Que hace cada bloque

PARTE	FUNCIÓN
%REQUIRE "3.7"	Exige Bison con counterexamples
LR.TYPE LALR	Parser LALR de la unidad
PARSE.ERROR DETAILED + LAC FULL	Mejores mensajes de error (no resuelven conflictos)
ALIAS ENTRE COMILLAS	Diagnósticos legibles ("+", "if", etc.)
PROGRAMA	Vacío o secuencia de sentencias
SENTENCIA	Asignación, if sin else, o if con else (aún ambigua)
EXPR	ID, NUM, +, *, paréntesis (sin %left todavía)

Construcción del leer (*lexer.l*)

Objetivo

Reconocer espacios, comentarios, palabras clave if/else, identificadores, números y operadores. Las palabras if y else deben ir antes de la regla de identificadores.

```
1   %option noyywrap nodefault yylineno noinput nounput never-interactive
2   %{
3   #include "parser.tab.h"
4   #include <stdio.h>
5   %}
6   LETRA   [A-Za-z_]
7   DIGITO  [0-9]
8   %%
9   [ \t\r\n]+                      { /* ignorar espacios */ }
10  "//"[^\n]*                      { /* ignorar comentario */ }
11  "if"                            { return SI; }
12  "else"                          { return SINO; }
13  {LETRA}({LETRA}|{DIGITO})*      { return ID; }
14  {DIGITO}+                       { return NUM; }
15  "="                             { return ASIGNA; }
16  "+"                             { return MAS; }
17  "*"                             { return POR; }
18  "("                             { return PAR_I; }
19  ")"                             { return PAR_D; }
20  ";"                             { return PUNTO_COMA; }
21  . {
22      fprintf(stderr, "Error lexico en la linea %d: caracter '%s'\n",
23              yylineno, yytext);
24      return (unsigned char) yytext[0];
25  }
26  %%
```

Ilustración 3 Código completo

Orden de las reglas (importante)

1. Espacios y comentarios → no devuelven token.
2. "if" y "else" → **antes** de {LETRA}..., si no Flex las clasificaría como ID.
3. Identificadores y números.
4. Operadores conocidos.
5. Regla . al final → carácter desconocido; informa error y lo devuelve al parser.

Opciones Flex

OPCIÓN	EFECTO
NOYYWRAP	Fin de archivo = fin de entrada
NODEFAULT	Evita ruta silenciosa para caracteres no cubiertos
YYLINENO	Actualiza el número de línea

NEVER-INTERACTIVE | Entrada por archivo/redirección, no terminal

Diagnostico inicial: 5 conflictos shift/reduce

Objetivo

Reproducir y documentar los conflictos sintácticos iniciales del parser antes de aplicar cualquier corrección. El objetivo es obtener exactamente 5 conflictos shift/reduce: 4 por la ambigüedad de los operadores (+ y *) y 1 por el problema del dangling else.

```
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ LC_
ALL=C bison -Wall -Wcounterexamples \
  --report=state,counterexamples \
  --report-file=evidencias/01_inicial.output \
  -d -o parser.tab.c parser.y \
  2> evidencias/01_inicial.txt
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ ls
-R
.:
README.md            evidencias  parser.tab.c  parser.y
ejecutar_pruebas.sh  lexer.l     parser.tab.h  pruebas

./evidencias:
00_versiones.txt  01_inicial.output  01_inicial.txt

./pruebas:
01_precedencia.txt  03_else_cercano.txt  05_operador_invalido.txt
02_parentesis.txt   04_else_doble.txt    06_else_sin_if.txt
```

Ilustración 4 Comandos ejecutados

Resultado esperado

- Se genera el archivo evidencias/01_inicial.output (la "radiografía" de estados y decisiones del parser).
- Se genera el archivo evidencias/01_inicial.txt (la bitácora de advertencias de Bison).
- Bison advierte **exactamente 5 shift/reduce conflicts**.

Cómo se distribuyen los conflictos:

1. Cuatro conflictos provienen de las combinaciones de operadores: +/+, +/*, */+ y */*.
2. Un conflicto proviene del else que podría asociarse a dos if anidados (*dangling else*).

Corrección de operadores: precedencia y asociatividad

Objetivo

Resolver los 4 conflictos shift/reduce generados por las combinaciones de los operadores + y *, reduciendo el total de conflictos de 5 a 1 (el conflicto restante será el *dangling else*).

```
%require "3.7"
%{
#include <stdio.h>

int yylex(void);
void yyerror(const char *mensaje);
extern int yylineno;
%}

%define lr.type lalr
%define parse.error detailed
%define parse.lac full

%token ID "identificador" NUM "numero"
%token SI "if" SINO "else"
%token ASIGNA "=" MAS "+" POR "*"
%token PAR_I "(" PAR_D ")" PUNTO_COMA ";"

/* Declaración de precedencia y asociatividad (Fase 5) */
/* La última declarada tiene mayor precedencia: POR > MAS */
/* %left indica asociatividad por la izquierda (ej. a+b+c se agrupa como (a+b)+c) */
%left MAS
%left POR

%start programa

%%
programa:
    %empty
  | programa sentencia
  ;

sentencia:
    ID ASIGNA expr PUNTO_COMA
  | SI PAR_I expr PAR_D sentencia
  | SI PAR_I expr PAR_D sentencia SINO sentencia
  ;

expr:
    ID
  | NUM
  | expr MAS expr
  | expr POR expr
  | PAR_I expr PAR_D
  ;
%%

void yyerror(const char *mensaje)
```

```
48  void yyerror(const char *mensaje)
49  {
50      fprintf(stderr, "Error sintactico en la linea %d: %s\n",
51              yylineno, mensaje);
52  }
53
54  int main(void)
55  {
56      return yyparse();
57  }
```

Ilustración 5 Código completo

```
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ Equ
ipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ LC_ALL
=C bison -Wall -Wcounterexamples \
  --report=state,solved,counterexamples \
  --report-file=evidencias/02_precedencia.output \
  -d -o parser.tab.c parser.y \
  2> evidencias/02_precedencia.txt
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$
-bash: Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflict
os$: No such file or directory
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ cat
 evidencias/02_precedencia.txt
-bash: Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflict
os$: No such file or directory
```

Ilustración 6 Comandos ejecutados

Corrección del dangling else: gramática emparejada/no emparejada

Objetivo

Eliminar el último conflicto shift/reduce reescribiendo la regla sentencia para distinguir entre sentencias que ya tienen todos sus else resueltos (sentencia_emparejada) y aquellas que aún tienen un if pendiente de cerrar (sentencia_no_emparejada). Esto obliga al parser a asociar el else con el if más cercano, eliminando la ambigüedad sin usar trucos como %expect.

```
%require "3.7"
%{
#include <stdio.h>

int yylex(void);
void yyerror(const char *mensaje);
extern int yylineno;
%}

%define lr.type lalr
%define parse.error detailed
%define parse.lac full

%token ID "identificador" NUM "numero"
%token SI "if" SINO "else"
%token ASIGNA "=" MAS "+" POR "*"
%token PAR_I "(" PAR_D ")" PUNTO_COMA ";"

%left MAS
%left POR

%start programa

%%
programa:
    %empty
  | programa sentencia
  ;

/* Reglas reescritas para resolver el dangling else (Fase 6) */
sentencia:
    sentencia_emparejada
  | sentencia_no_emparejada
  ;

sentencia_emparejada:
    ID ASIGNA expr PUNTO_COMA
  | SI PAR_I expr PAR_D sentencia_emparejada SINO sentencia_emparejada
  ;

sentencia_no_emparejada:
    SI PAR_I expr PAR_D sentencia
  | SI PAR_I expr PAR_D sentencia_emparejada SINO sentencia_no_emparejada
  ;

expr:
    ID
  | NUM
```

```
48    | NUM
49    | expr MAS expr
50    | expr POR expr
51    | PAR_I expr PAR_D
52    ;
53  %%
54
55  void yyerror(const char *mensaje)
56  {
57      fprintf(stderr, "Error sintactico en la linea %d: %s\n",
58              yylineno, mensaje);
59  }
60
61  int main(void)
62  {
63      return yyparse();
64  }
```

Ilustración 7 Código completo

```
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ LC_
ALL=C bison -Wall -Wcounterexamples \
  --report=state,solved,counterexamples \
  --report-file=evidencias/03_else_resuelto.output \
  -d -o parser.tab.c parser.y \
  2> evidencias/03_else_resuelto.txt
```

Ilustración 8 Comandos ejecutados

Validación estricta y compilación final

Objetivo

Convertir cualquier conflicto sintáctico pendiente en error de compilación mediante las banderas -Werror=conflicts-sr y -Werror=conflicts-rr de Bison, y compilar el parser final con GCC en modo estricto (C11, todas las advertencias activas y tratadas como errores). Esto garantiza que el entregable no tenga ambigüedades ocultas.

parser.y U | lexer.l U

Ubuntu > home > Equipo1 > compint > actividades > actividad05_conflictos > lexer.l

```
1   %option noyywrap nodefault yylineno noinput nounput never-interactive
2   %{
3   #include "parser.tab.h"
4   #include <stdio.h>
5   %}
6   LETRA   [A-Za-z_]
7   DIGITO  [0-9]
8   %%
9   [ \t\r\n]+                      { /* ignorar espacios */ }
10  "//"[^\n]*                      { /* ignorar comentario */ }
11  "if"                            { return SI; }
12  "else"                          { return SINO; }
13  {LETRA}({LETRA}|{DIGITO})*      { return ID; }
14  {DIGITO}+                       { return NUM; }
15  "="                             { return ASIGNA; }
16  "+"                             { return MAS; }
17  "*"                             { return POR; }
18  "("                             { return PAR_I; }
19  ")"                             { return PAR_D; }
20  ";"                             { return PUNTO_COMA; }
21  . {
22      fprintf(stderr, "Error lexico en la linea %d: caracter '%s'\n",
23              yylineno, yytext);
24      return (unsigned char) yytext[0];
25  }
26  %%
```

parser.y U | lexer.l U

Ubuntu > home > Equipo1 > compint > actividades > actividad05_conflictos > parser.y

```
%require "3.7"
%{
#include <stdio.h>

int yylex(void);
void yyerror(const char *mensaje);
extern int yylineno;
%}

%define lr.type lalr
%define parse.error detailed
%define parse.lac full

%token ID "identificador" NUM "numero"
%token SI "if" SINO "else"
%token ASIGNA "=" MAS "+" POR "*"
%token PAR_I "(" PAR_D ")" PUNTO_COMA ";"

%left MAS
%left POR

%start programa

%%
programa:
    %empty
  | programa sentencia
  ;

sentencia:
    sentencia_emparejada
  | sentencia_no_emparejada
  ;

sentencia_emparejada:
    ID ASIGNA expr PUNTO_COMA
  | SI PAR_I expr PAR_D sentencia_emparejada SINO sentencia_emparejada
  ;

sentencia_no_emparejada:
    SI PAR_I expr PAR_D sentencia
  | SI PAR_I expr PAR_D sentencia_emparejada SINO sentencia_no_emparejada
  ;

expr:
    ID
  | NUM
  | expr MAS expr
```

```
48      | expr MAS expr
49      | expr POR expr
50      | PAR_I expr PAR_D
51      ;
52  %%
53
54  void yyerror(const char *mensaje)
55  {
56      fprintf(stderr, "Error sintactico en la linea %d: %s\n",
57              yylineno, mensaje);
58  }
59
60  int main(void)
61  {
62      return yyparse();
63  }
```

Ilustración 9 Códigos completos

```
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ LC_ALL=C bison -Wall \
  -Werror=conflicts-sr -Werror=conflicts-rr \
  --report=state,solved \
  --report-file=evidencias/04_final.output \
  -d -o parser.tab.c parser.y \
  2> evidencias/04_final.txt

echo "Código de salida de Bison: $?"
Código de salida de Bison: 0
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ flex -o lex.yy.c lexer.l
echo "Código de salida de Flex: $?"
Código de salida de Flex: 0
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ gcc -std=c11 -Wall -Wextra -Wpedantic -Werror \
  parser.tab.c lex.yy.c -o parser

echo "Código de salida de GCC: $?"
Código de salida de GCC: 0
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ echo "=== Verificación final ==="
echo "Líneas en 04_final.txt: $(wc -l < evidencias/04_final.txt)"
ls -l parser
./parser < /dev/null
echo "Código de salida al ejecutar parser vacío: $?"
=== Verificación final ===
Líneas en 04_final.txt: 0
-rwxr-xr-x 1 Equipo1 Equipo1 36184 Nov  6 23:13 parser
Código de salida al ejecutar parser vacío: 0
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$
```

Ilustración 10 Comandos ejecutados

Pruebas obligatorias: validación del parser

Objetivo

Verificar que el parser acepta correctamente las entradas válidas (código de salida 0) y rechaza las entradas inválidas (código de salida 1), confirmando que la gramática resuelta funciona como se espera.

Archivos de prueba

pruebas/01_precedencia.txt

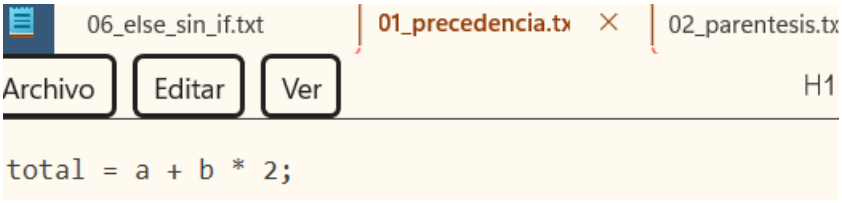


Ilustración 11 Contenido

pruebas/02_parentesis.txt

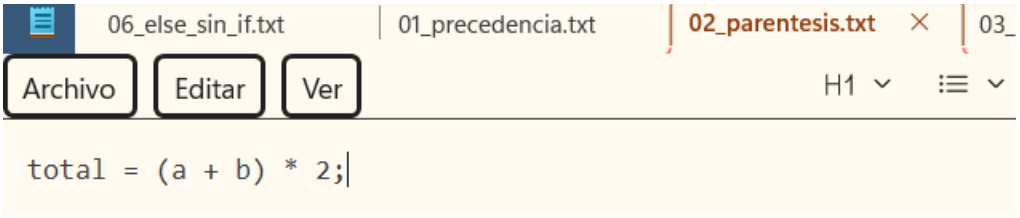


Ilustración 12 Contenido

pruebas/03_else_cercano.txt

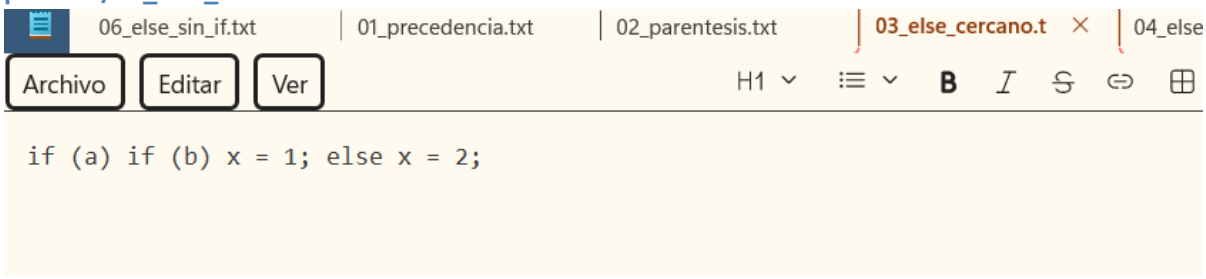


Ilustración 13 Contenido

pruebas/04_else_doble.txt

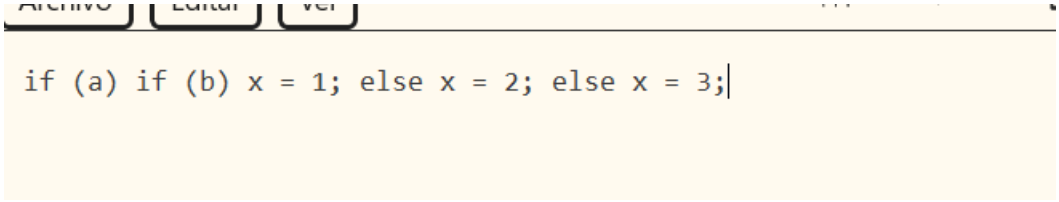


Ilustración 14 Contenido

pruebas/05_operador_invalido.txt

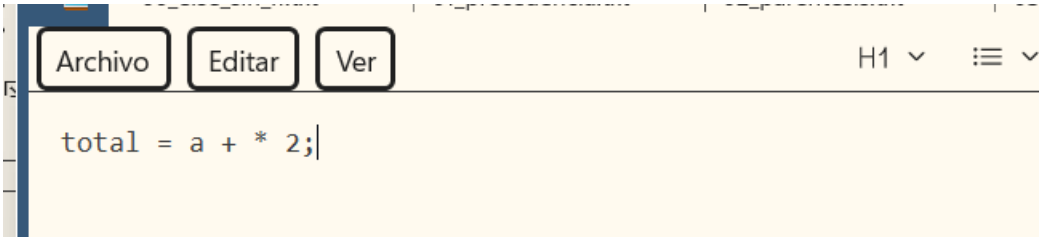


Ilustración 15 Contenido

pruebas/06_else_sin_if.txt

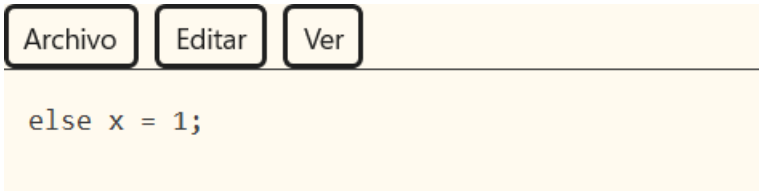


Ilustración 16 Contenido

Ubuntu > home > Equipo1 > compint > actividades > actividad05_conflictos > ejecutar_pruebas.sh

```
1   #!/usr/bin/env bash
2   set -u
3
4   fallos=0
5
6   probar() {
7     archivo="$1"
8     esperado="$2"
9     if ./parser < "$archivo" >/dev/null 2>&1; then
10      obtenido=0
11    else
12      obtenido=$?
13    fi
14    if [ "$obtenido" -eq "$esperado" ]; then
15      echo "OK  $archivo -> $obtenido"
16    else
17      echo "FALLO  $archivo -> $obtenido; se esperaba $esperado"
18      fallos=$((fallos + 1))
19    fi
20  }
21
22  probar pruebas/01_precedencia.txt 0
23  probar pruebas/02_parentesis.txt 0
24  probar pruebas/03_else_cercano.txt 0
25  probar pruebas/04_else_doble.txt 0
26  probar pruebas/05_operador_invalido.txt 1
27  probar pruebas/06_else_sin_if.txt 1
28
29  test "$fallos" -eq 0
```

Ilustración 17 Código completo de ejecutar_pruebas.sh

```
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ set
 -o pipefail
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ ./e
jecutar_pruebas.sh | tee evidencias/05_pruebas.txt
OK  pruebas/01_precedencia.txt -> 0
OK  pruebas/02_parentesis.txt -> 0
OK  pruebas/03_else_cercano.txt -> 0
OK  pruebas/04_else_doble.txt -> 0
OK  pruebas/05_operador_invalido.txt -> 1
OK  pruebas/06_else_sin_if.txt -> 1
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$ ech
o "Código de salida del script: $?"
Código de salida del script: 0
Equipo1@TechNova-Group:~/compint/actividades/actividad05_conflictos$
```

Ilustración 18 Comandos ejecutados